



LMS Platform: Master Interview Guide

Complete Technical Architecture & Technical Interview Preparation Handbook

MERN Stack

React 18 + Vite

Redux Toolkit

Node.js + Express

MongoDB & Mongoose

Razorpay Payment Engine

Cloudinary Media CDN

JWT HTTP-Only Auth

1. Executive Project Overview

The **Learning Management System (LMS) Platform** is a full-stack web application designed to support online education delivery, video streaming, role-based administration, and recurring subscription monetization. It features a responsive React single-page application (SPA) frontend powered by Vite and Redux Toolkit, backed by an Express RESTful API and MongoDB database.

💡 Elevator Pitch Strategy for Interviewers

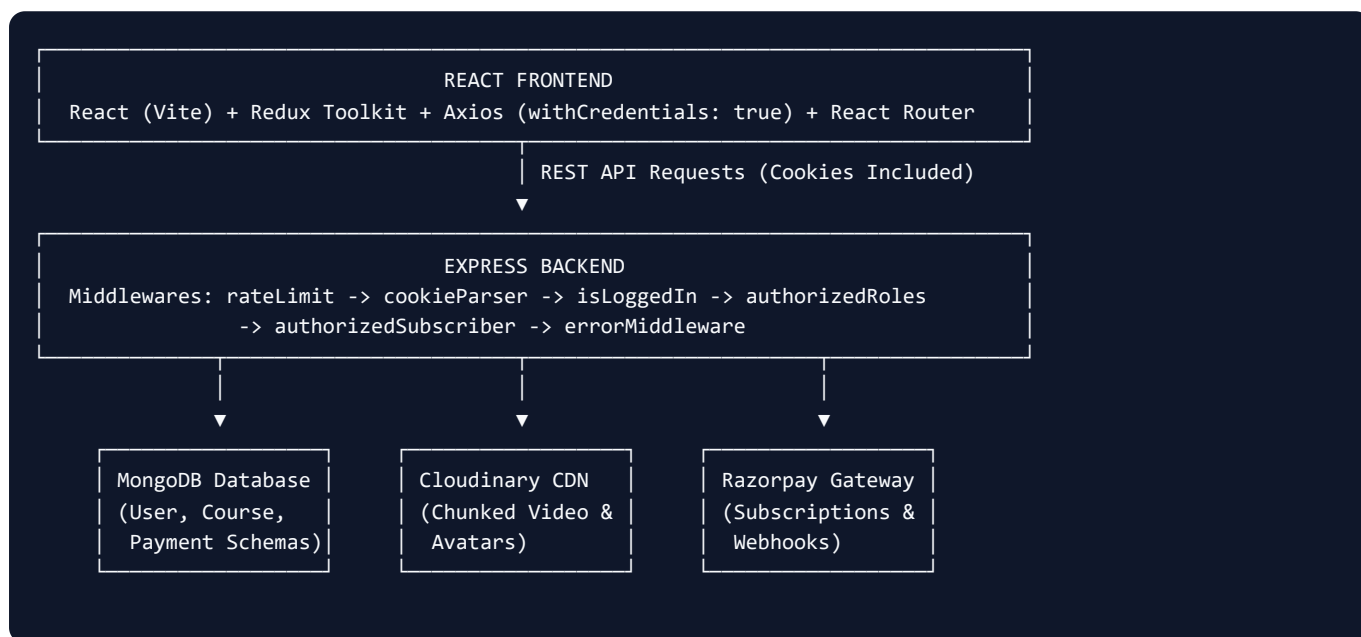
"I built a full-stack Learning Management System (LMS) designed for online course hosting and subscription monetization. On the frontend, I used React with Vite for high performance, Redux Toolkit for centralized state management, and Tailwind CSS for dynamic dark-themed UI components. On the backend, I engineered a Node.js and Express REST API with MongoDB/Mongoose. Key highlights include HTTP-Only cookie-based JWT authentication, role and subscription access control middlewares, Razorpay subscription processing with HMAC SHA-256 signature verification, Cloudinary chunked media streaming, and an Admin analytics dashboard powered by Chart.js."

Tech Stack Breakdown

Layer	Technologies Used	Role & Architectural Purpose
Frontend	React 18, Vite, Tailwind CSS, DaisyUI	Fast SPA rendering, modern UI styling, responsive design.
State & Routing	Redux Toolkit, React Router DOM v6	Global state slices (auth, course, lecture, razorpay, stat), role routing.
Backend API	Node.js, Express.js	REST API controller pattern, middleware pipeline, CORS setup.
Database	MongoDB, Mongoose ORM	NoSQL document database, schema validation, pre-save hooks.
Authentication	JWT (HTTP-Only Cookies), Bcrypt.js, Crypto	Secure authentication tokens, salted password hashing, password reset tokens.

Layer	Technologies Used	Role & Architectural Purpose
Payment Gateway	Razorpay API, Webhooks	Subscription plans, HMAC-SHA256 signature verification, billing event hooks.
Media Storage	Multer, Cloudinary SDK	Local disk buffer upload, 6MB chunked upload (`upload_large`) for HD video.
Security & Utilities	express-rate-limit, Nodemailer, Morgan	IP brute-force protection (20 req/15 min), transaction emails, dev logging.

System Architecture & Data Workflow



2. Key Functional Modules & Technical Nuances

A. Authentication & Security Engine

- **HTTP-Only Cookie Token Storage:** Access tokens are issued with `httpOnly: true`, `secure: process.env.NODE_ENV === 'production'`, and a 7-day expiration duration, eliminating client-side XSS access to tokens.
- **Bcrypt Password Hashing:** Mongoose `pre('save')` hook automatically hashes user passwords using salt rounds = 10 when modified. Passwords are set with `select: false` in schemas to avoid accidental leaks in database queries.
- **Crypto Reset Tokens:** Password reset link uses Node's native `crypto.randomBytes(20)`. Only an SHA-256 hash of the token is saved to the database (expires in 5 minutes), while the unhashed token is emailed to the user.
- **IP Rate Limiting:** Endpoints like `/login`, `/register`, and `/forgot` are protected with `express-rate-limit` to limit abuse to 20 attempts per 15 minutes per IP address.

B. Authorization & Protected Routes (RBAC & Subscription Guard)

- **isLoggedIn Middleware:** Decodes and validates JWT tokens from cookies using `jwt.verify()`.
- **authorizedRoles('ADMIN') Middleware:** Restricts route execution to users with the `ADMIN` role.
- **authorizedSubscriber Middleware:** Grants lecture video access only if the user is an `ADMIN` or has an `active` subscription status.
- **Frontend RequireAuth Guard:** Custom React wrapper component inspecting Redux state to enforce role routes and navigate unauthorized users to a `/denied` route.

C. Media Upload & CDN Streaming Pipeline

- **Multer Storage:** Incoming files are temporarily cached on local disk storage under the `uploads/` directory.
- **Cloudinary Chunked Uploads:** Large lecture video files are streamed to Cloudinary using `cloudinary.v2.uploader.upload_large()` with a `chunk_size: 6000000` (6MB chunks).
- **Disk Cleanup:** After successful CDN upload, temporary files are unlinked using `fs.rm()`.

D. Razorpay Payment & Subscription Engine

- **Subscription Plan Creation:** Users subscribe to an admin-configured subscription plan ID.
- **HMAC SHA-256 Signature Verification:** Server verifies subscription payments by hashing `razorpay_payment_id|subscription_id` with `RAZORPAY_SECRET`.
- **Automated Webhook Sync:** Webhooks catch subscription billing events (e.g., `subscription.charged`) to maintain accurate billing state asynchronously.

E. Admin Analytics & Reporting

- **User Breakdown:** Interactive Pie chart displaying registered vs active subscribed users.
- **Revenue Graph:** Interactive Bar chart plotting monthly revenue trends across the current calendar year.

3. Top Technical & Behavioral Interview Questions

SECURITY Q1: Why did you use HTTP-Only cookies instead of LocalStorage for JWT tokens?

Answer: Storing JWTs in `localStorage` or `sessionStorage` makes them accessible to any client-side JavaScript executing in the browser. If the web application is vulnerable to Cross-Site Scripting (XSS), malicious scripts can easily steal the raw token and impersonate the user.

By issuing JWTs in `httpOnly` cookies with `secure: true` and `sameSite` flags, client-side scripts cannot read or steal the token. The browser automatically appends the cookie to requests made to the backend domain.

PAYMENTS Q2: How does Razorpay payment verification work in your server?

Answer: Payment verification relies on a cryptographic HMAC-SHA256 checksum:

1. The client completes payment via Razorpay SDK and receives `razorpay_payment_id`, `razorpay_subscription_id`, and `razorpay_payment_signature`.
2. The server recalculates the signature using Node's `crypto` module: `crypto.createHmac('sha256', process.env.RAZORPAY_SECRET).update(`${razorpay_payment_id}|${subscriptionId}`).digest('hex')`.
3. If the generated string matches the incoming signature, the subscription status is updated to `active` and saved in MongoDB.

FRONTEND Q3: How do you maintain authenticated requests with CORS in React + Axios?

Answer: Cross-origin requests do not transmit cookies by default. To enable secure cookie transfer across origins:

- **Backend:** In `app.js`, Express CORS middleware is configured with `credentials: true` and an explicit `origin` domain whitelist.
- **Frontend:** The global Axios instance is initialized with `withCredentials: true`.

PERFORMANCE**Q4: How do you handle multi-gigabyte video uploads without crashing Node.js?**

Answer: Direct memory buffers in Express easily lead to heap out-of-memory errors when processing large video files. I solved this using a two-stage streaming upload process:

1. **Multer Disk Caching:** Multer streams incoming binary video streams directly onto the disk inside a temporary `uploads/` directory instead of loading the entire file into Node RAM.
2. **Cloudinary Chunked Uploads:** I implemented Cloudinary's `upload_large` API with a 6MB chunk size (`chunk_size: 6000000`). Cloudinary reads and uploads the file in small chunks.
3. **Cleanup:** Once uploaded, `fs.rm()` removes the local disk buffer.

SYSTEM SCALING**Q5: How would you scale this platform to support 100,000 active students?**

Answer: To scale the architecture seamlessly:

- **HLS Video Streaming:** Convert MP4 videos to HLS (HTTP Live Streaming) format with adaptive bitrates delivered via an edge CDN (AWS CloudFront / Cloudinary).
- **Caching Layer:** Deploy a **Redis** cluster to cache user profile sessions, subscription statuses, and course catalog metadata to minimize database hits.
- **Stateless Horizontally Scaled Microservices:** Because JWT auth is stateless, backend Express containers can run in Kubernetes behind an AWS Application Load Balancer (ALB).
- **Database Optimization:** Create indexes on `email`, `role`, and `subscription.status`, and introduce MongoDB read-replicas or sharding by `userId`.

BEHAVIORAL**Q6: What was a major bug you encountered and how did you debug it?**

Answer: A key bug occurred when subscribed users were redirected back to the login page after purchasing a subscription. The issue was that the original JWT token stored in the cookie contained the stale `subscription.status = 'inactive'` payload. Even though MongoDB updated to `active`, the client's cookie held the old token.

Fix: Inside the `verifySubscription` controller, after saving the updated user document to MongoDB, I generated a fresh JWT token with the new subscription status and re-issued the HTTP cookie before returning the success response.

4. ⚡ 10-Minute Pre-Interview Rapid Recall Cheat Sheet

🚀 Last-Minute Memory Refresh

Review these bullet points 10 minutes prior to your interview call:

- **Frontend Stack:** React 18, Vite, Redux Toolkit, Tailwind CSS, DaisyUI, React Router v6, Chart.js.

- **Backend Stack:** Node.js, Express.js, MongoDB, Mongoose, JWT, Bcrypt, Multer, Nodemailer.
- **Auth Strategy:** HTTP-Only Cookies + JWT + Bcrypt (salt rounds=10) + Rate Limiting (`express-rate-limit`).
- **Password Reset:** Raw random token via email, SHA-256 token stored in DB with 5-minute expiration timestamp.
- **Access Control:** `isLoggedIn` $\rightarrow$ `authorizedRoles('ADMIN')` $\rightarrow$ `authorizedSubscriber` .
- **Razorpay Sync:** HMAC-SHA256 signature verification (`razorpay_payment_id|subscription_id`) + Razorpay Webhooks.
- **Media Storage:** Multer disk caching $\rightarrow$ Cloudinary 6MB chunked upload (`upload_large`) $\rightarrow$ `fs.rm()` cleanup.
- **Dashboard Analytics:** Chart.js Pie chart (users) & Bar chart (monthly sales revenue).